Laboratorul 6
Controlul versiunilor

1 Programe și versiuni

În cadrul ciclului uzual de dezvoltare a programelor apare adesea nevoia de a gestiona mai multe versiuni ale aceluiași program. Fie că e vorba de încercarea de a schimba o variantă stabilă a unui program, încercare care într-un fel sau altul eșuează și e nevoie să revenim la varianta stabilă, fie că se dorește păstrarea ambelor variante, apare necesitatea de a gestiona versiuni multiple ale aceluiași program de-a lungul evoluției sale.

Acest proces, cunoscut sub numele de *Controlul Versiunilor* sau *Revision Control* (caz în care versiunile se numesc *revizii*), reprezintă gestiunea mai multor versiuni ale aceluiași fișier (sau, în general, a unei colecții de fișiere). O revizie poate avea semnificații multiple, de la o versiune scoasă pe piață (un așa numit *release*) până la rezolvarea unui defect (*bug fixing*) sau adăugarea de funcționalități.

Procesul are deopotrivă avantaje și dezavantaje. Avantajele țin de posibilitatea de a naviga printre revizii înainte și înapoi, de a identifica mai repede momentul apariției unui defect, lucrul în echipă ș.a.m.d. Dezavantajele sunt legate în principal de complexitatea sporită a procesului, care necesită lucru suplimentar față de programarea propriu-zisă și o organizare suplimentară a dezvoltării programelor.

Noțiunea centrală în Revision Control este *repository*-ul. Acesta este colecția tuturor versiunilor fișierelor și directoarelor dintr-un proiect. Uzual, un repository este stocat pe un server de unde utilizatorii îl pot accesa. Modul lor de lucru uzual implică crearea unei copii locale a repository-ului (operație numită *checkout*), modificarea unuia sau a mai multor fișiere și transmiterea modificărilor către server (operația de *commit*). Pe cale de consecință, serverul creează o nouă versiune a proiectului.

Această secvență tipică de utilizare a repository-urilor poate fi mai complicată atunci când repository-ul este folosit de mai mulți programatori care

lucrează în echipă. Modificările făcute de unul dintre membrii echipei pot fi văzute de ceilalți membri atunci când își actualizează copia locală a repository-ului prin operația de *update*. Cu ocazia unui update se întâmplă adesea ca modificările operate să genereze conflicte. Într-o primă fază, acestea sunt rezolvate automat, iar dacă rezoluția nu este posibilă, se recurge la intervenția manuală a utilizatorului.

Situații și mai complicate se înregistrează atunci când apar versiuni complet noi în repository care deviază de la ramura principală de dezvoltare a proiectului, numită și *trunk*. Aceste bifurcații se numesc ramuri sau *branch*-uri. Integrarea branch-urilor implică o procedură de *merge* care poate genera conflicte, caz în care se aplică procedura de rezoluție a conflictelor, sau nu, caz în care merge-ul reprezintă pur și simplu o nouă modificare în program.

2 Comenzi

În acest laborator vom folosi `git(1)` pentru a învăța lucrul cu controlul versiunilor. Pentru a inițializa un repository nou folosiți subcomanda `init`:

```
$ mkdir testrepo
$ cd testrepo
$ git init
```

Toate datele necesare funcționării repository-ului se găsesc în directorul `.git`. Un prim pas este să descrieți scopul noului proiect:

```
$ vi .git/description
```

și să vă adăugați datele personale care vor fi folosite când faceți un commit:

```
$ git config user.name "Alex Alexandrescu"
$ git config user.email "alex@gmail.com"
```

Dacă doriți ca aceste date să fie folosite pentru fiecare repository de pe calculator adăugați opțiunea `--global` după `config`.

Conținutul nou, fișiere și directoare, trebuie întâi adăugat în lista de fișiere urmărite de repository cu subcomanda `add`:

```
$ git add myfile.c
```

și pe urmă făcut `commit` cu tot ce s-a schimbat (inclusiv noile fișiere adăugate):

```
$ git commit myfile.c
```

Această comandă va porni editorul pentru a completa o descriere a schimbărilor aduse de acest commit. Completați, salvați și ieșiți din editor. Commit-ul este efectuat.

Pentru a trimite schimbările către alte repository-uri se folosește subcomanda `push`. Întâi trebuie adăugată o intrare pentru fiecare repository cu care vrem să comunicăm folosind subcomanda `remote`:

```
$ git remote add origin https://github.com/user/repo.git
```

În exemplu, `origin` este alias-ul repository-ului. Tradițional, acest nume este rezervat remote-ului către care se va face implicit `push`:

```
$ git push origin
```

Pentru a prelua schimbările de la un alt repository se folosește similar subcomanda `pull`:

```
$ git pull origin
```

Este recomandat, pentru a evita pe cât posibil operațiile de `merge`, să se folosească opțiunea `--rebase`, care va evita un commit de tip `merge` dacă este posibil și nu apar conflicte:

```
$ git pull --rebase origin
```

Pentru a copia (sau clona) un repository existent folosiți subcomanda `clone`:

```
$ git clone https://github.com/user/repo.git
```

Protocolul de comunicare poate fi `git`, `ssh`, `http` sau `ftp` în funcție de portul ales de gazdă. Implicit se folosește `ssh`.

Pentru a crea o ramură nouă se folosește subcomanda `branch`:

```
$ git branch newtopic
```

Ramura principală se numește `master` în `git(1)`. Pentru a schimba ramura folosiți comanda `checkout`:

```
$ git checkout newtopic
$ git checkout master
```

Alte subcomenzi uzuale:

- `diff` – arată modificările necomise în format `diff(1)`;
- `log` – arată jurnalul commit-urilor;
- `show` *`commit-id`* – arată descrierea și schimbările aduse de un commit.

3 Sarcini de laborator

1. Creați-vă un cont pe GitHub și adăugați-vă cheia publică (*Settings* → *SSH and GPG keys*).

2. Creați un repository nou pe GitHub numit `hosts` și clonați-l local. Local, configurați-vă numele și adresa de mail și scrieți o descriere scurtă a repository-ului.

3. În repository-ul local `hosts` creați două commit-uri: primul cu scriptul care verifică validitatea adreselor IP din `/etc/hosts` pe care l-ați dezvoltat în laboratorul trecut. Al doilea commit îl faceți cu varianta modificată a scriptului respectiv, care folosește o funcție shell pentru verificarea validității unei adrese IP. Funcția primește ca parametri un nume de host și o adresă IP și verifică asocierea folosind un server DNS furnizat ca al treilea parametru al funcției.

4. Trimiteți schimbările făcute în repository-ul local către cel de pe GitHub.

5. Grupați-vă doi câte doi: studentul A împreună cu studentul B. Studentul A face un *fork* al repository-ului lui B de pe GitHub. În acest nou repository, A face un nou commit (de exemplu adaugă o linie în plus care afișează numele studentului A), după care tot A face un *pull request* către B. B acceptă această cerere. Cum arată cele două repository-uri acum?